

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 13	CLOs to be covered: CLO2
Lab Title: MAP, Filter and Reduce	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
-------	-------	-------	--------

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 13

Lab Goals / Objectives:

- Understand functional programming concepts in Python
- Master the map() function for transforming iterables
- Master the filter() function for selecting elements
- Learn the reduce() function from the functools module
- Combine lambda functions with map, filter, and reduce
- Differentiate between map/filter/reduce and list comprehensions
- Apply functional programming techniques to practical problems

Equipment Required:

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory:

1. Functional Programming in Python

Functional programming is a programming paradigm where computation is treated as the evaluation of mathematical functions. Python supports functional programming through tools like `map()`, `filter()`, `reduce()`, and `lambda` functions. These tools allow writing cleaner, more declarative code.

2. The `map()` Function

The `map()` function applies a specified function to every item of an iterable (like a list) and returns an iterator with the results. It is one of the most commonly used functional programming tools in Python.

Syntax: `map(function, iterable)`

```
numbers = [1, 2, 3, 4]
squares = list(map(lambda x: x**2, numbers))
# Result: [1, 4, 9, 16]
```

The `map()` function is more concise than writing an explicit for loop and is particularly useful when the transformation logic is simple.

3. The `filter()` Function

The `filter()` function constructs an iterator from elements of an iterable for which a function returns `True`. It is used to select elements that satisfy a specific condition.

Syntax: `filter(function, iterable)`

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
# Result: [2, 4, 6]
```

The function passed to `filter()` should return a boolean value. Only elements for which the function returns `True` are included in the result.

4. The reduce() Function

The `reduce()` function from the `functools` module applies a rolling computation to sequential pairs of values in an iterable. It reduces the iterable to a single value.

Syntax: `reduce(function, iterable, initializer)`

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
# Result: 120 (1*2*3*4*5)
```

`reduce()` is useful for aggregations like summing all elements, finding the product, or finding the maximum/minimum.

5. functools Module

The `functools` module provides higher-order functions that act on or return other functions. Key functions include:

- `reduce()` - Rolling computation on a list
- `partial()` - Creates a partial function with fixed arguments
- `wraps()` - Decorator for preserving function metadata

6. map() vs List Comprehension

Both `map()` and list comprehensions can transform iterables. List comprehensions are generally preferred in Python for readability:

```
# Using map
squares = list(map(lambda x: x**2, numbers))

# Using list comprehension (preferred)
squares = [x**2 for x in numbers]
```

However, `map()` can be more efficient for large datasets and when the transformation function is already defined.

7. Combining map() and filter()

map() and filter() can be combined for powerful data processing pipelines:

```
numbers = [1, 2, 3, 4, 5, 6]
# Square only even numbers
result = list(map(lambda x: x**2, filter(lambda x: x % 2 == 0, numbers)))
# Result: [4, 16, 36]
```

8. Practical Applications

- map(): Data transformation, type conversion, normalization
- filter(): Data cleaning, selecting valid records, removing outliers
- reduce(): Aggregations, cumulative calculations, finding extremes

Lab Work:

Task 1: Square Function

Write a regular function to calculate the square of a number.

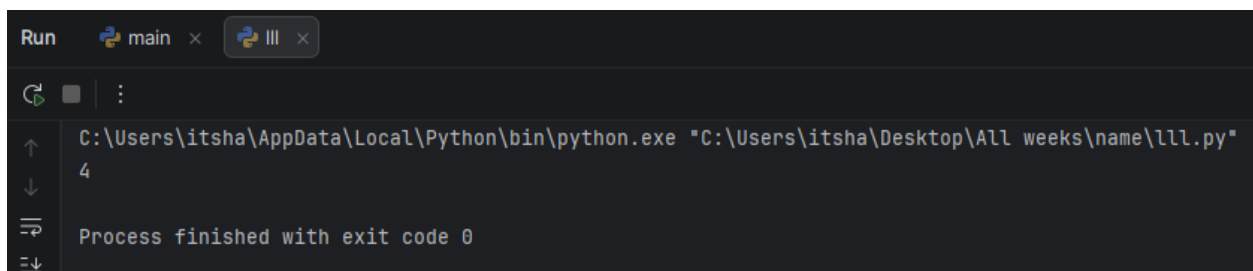
Code Summary:

This task establishes the foundation by defining a regular function that will be compared with lambda and map() approaches.

Key Code Lines:

```
# Square Function
def square(number):
    return number ** 2
print(square(2))
```

Output:



```
Run  main x  III x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\lll.py"
4
Process finished with exit code 0
```

Task 2: Lambda Square and Multiplication

Write lambda functions for squaring and multiplication operations.

Code Summary:

This task demonstrates using lambda for simple mathematical operations, showing the conciseness of lambda syntax compared to regular functions.

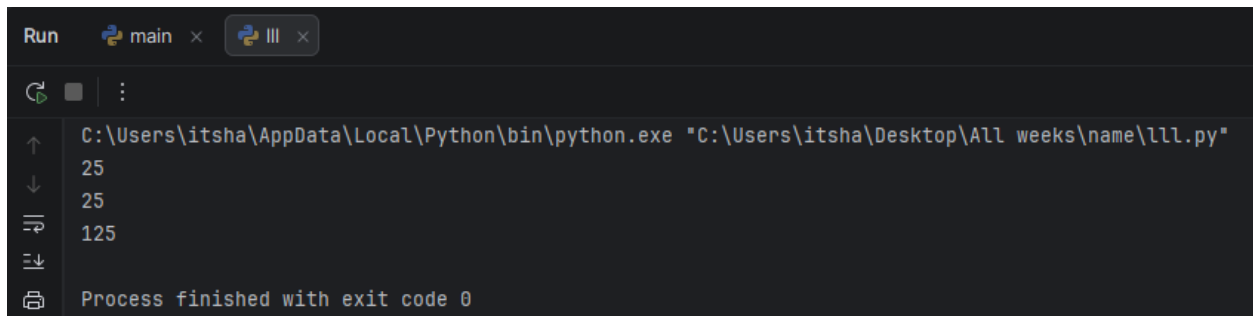
Key Code Lines:

```
# Lambda Square and Multiplication
a = lambda x: x * x
print(a(5))

a = lambda x, y: x * y
print(a(5, 5))

a = lambda x, y, z: x * y * z
print(a(5, 5, 5))
```

Output:



```
Run  main x  III x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\lll.py"
25
25
125
Process finished with exit code 0
```

Task 3: map() with Lambda

Use the map() function with a lambda to square all elements in a list.

Code Summary:

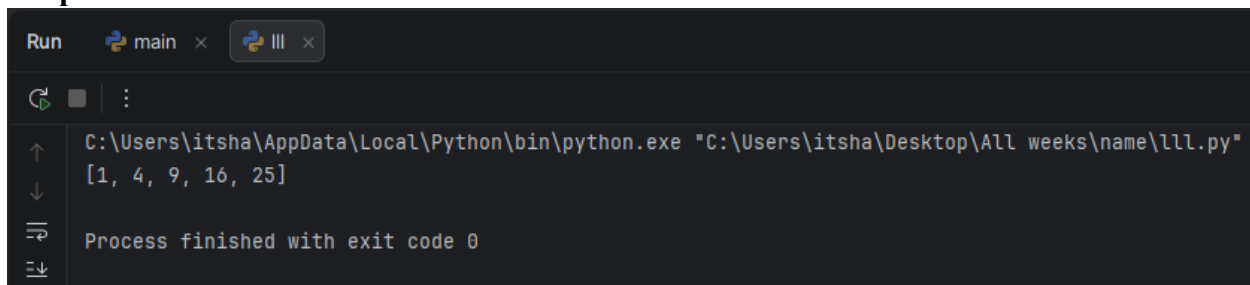
This task demonstrates the functional programming approach: applying a transformation to every element of a collection without explicit loops.

Key Code Lines:

```
# map() with Lambda
x = [1, 2, 3, 4, 5]
```

```
a = list(map(lambda x: x * x, x))
print(a)
```

Output:

A screenshot of a Python IDE window titled 'main'. The console shows the command prompt path 'C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\l11.py"' followed by the output '[1, 4, 9, 16, 25]'. Below the output, it says 'Process finished with exit code 0'. The IDE interface includes a 'Run' button and a file explorer on the left.

Code Summary:

This task shows how `filter()` selects elements from a list based on a condition, returning only those elements that satisfy the predicate.

Key Code Lines:

```
# filter() with Lambda
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)
```

Output:

A screenshot of a Python IDE window titled 'main'. The console shows the command prompt path 'C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\l11.py"' followed by the output '[2, 4, 6, 8, 10]'. Below the output, it says 'Process finished with exit code 0'. The IDE interface includes a 'Run' button and a file explorer on the left.

Task 5: reduce() from functools

Use the `reduce()` function to calculate the product of all numbers in a list.

Code Summary:

This task demonstrates `reduce()` for aggregation. The lambda takes two arguments and applies the operation cumulatively from left to right.

Key Code Lines:

```
# reduce() from functools
from functools import reduce
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
print("Product:", product)

# Finding maximum using reduce
maximum = reduce(lambda x, y: x if x > y else y, numbers)
print("Maximum:", maximum)
```

Output:



```
Run  main ×  III ×
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\111.py"
Product: 120
Maximum: 5
Process finished with exit code 0
```

Conclusions:

- Successfully understood functional programming concepts in Python
- Mastered the map() function for transforming all elements of an iterable
- Mastered the filter() function for selecting elements based on conditions
- Learned the reduce() function for aggregating iterables to a single value
- Combined lambda functions with map(), filter(), and reduce() for concise code
- Understood the differences and trade-offs between functional and imperative approaches
- Applied functional programming techniques to solve practical problems